

balancing mvls

from one to the other

bachotex 2025 meeting

Disclaimer

This is work in progress. We hope to be stable around the ConT_EXt meeting so what we give here is an impression. The primitives shown are meant to be used in mechanisms that bring them together.

In this talk we go:

- from paragraphs
- via pages
- and columns
- to mvls
- as well as balancing
- and discuss what is involved

What follow is mostly an outline for myself.

The context

- In LuaMetaTeX we have an extended, multi-pass par builder, the mechanism that breaks lines into paragraphs.
- We also extended the page builder a bit, the mechanism that collects and split of content.
- Alongside these we added a new mechanism that relates these two in terms of solution space.

Paragraphs

In the par builder the engine deals with:

- orphan penalties : lone words on the last line
- toddler penalties : single glyphs at the margin
- twin penalties : similar words at the margin
- spacing glue : glue between words
- punctuation glue : glue around punctuation
- l r m boxes : repeated content at breaks
- various math : all kind of constraints
- widow penalties : between last lines
- club penalties : between first lines
- shape penalties : between shape lines
- spacing (stretch & shrink) : criteria driven
- emergency stretch : just in case
- left & right margins : determines width
- (user) leaders : adaptive content
- glyph expansion : stretching and shrinking
- character protrusion : escaping into margins
- discretionaries : end and begin line snippets
- hyphenation (penalties) : breaks within words
- optional content : multi-pass wiggle room
- shapes & indentation : determines width
- broken penalties : between hyphenated lines
- interline penalties : between lines
- left & right hanging : determines width
- initial left & right : determines width
- final left & right : determines width

Pages

In the page builder the engine deals with:

- vsize : the main constraint
- inserts : need to be bound
- glue : can ease constraints
- penalties : drives the routine
- adjusted content : used for special purposes
- marks : synchronized content
- vz wiggle room : less constraints
- top & bottom skip : for aligning the bottom
- extra goal : optional wiggle room
- feedback loop : permits column construction

Columns

See Mikael's presentation. We have different solutions for different situations:

- **Simple columns:** Collect content in a vbox and let $\text{T}_{\text{E}}\text{X}$ do the splitting.
- **Mixed columns:** Set the vsize to n times the text height and when we have a full page split the collected content. This is sensitive for spacing and decisions around page and column breaks. The splitting is mostly delegated to Lua. (**Boxed columns** are a variant of this.)
- **Page columns:** Set the vsize to the text height and collect pages as usual but delay flushing till the number of columns is reached. The splitting is mostly done in $\text{T}_{\text{E}}\text{X}$ using the regular builder.
- **Column sets (old):** Change the vsize every time a pseudo page is split off. In MkIV we let Lua do some of the work.
- **Column sets (new):** Collect content and define page template and at some point flush the content to fit those templates using multi-slot optimization.

Main vertical lists

- In vertical mode the engine is either assembling a vbox (or equivalent like an insert) or adding to the main vertical list.
- Splitting of pages from that vertical list happen as we go.
- The vsplit routine is not the same as the page split routine.
- In ConT_EXt we migrate deeply hidden material (inserts, marks, adjusts) to the outer level (in Lua-MetaT_EX natively or MkIV via callbacks).
- In LuaMetaT_EX we can (efficiently) collect to different vertical lists without packaging (vboxing) or splitting (page break).

Balancing

- This evolved out of a previous (MkIV/MkXL) columnset implementation.
- We started with a similar approach as the par builder: choosing the best solution out of (possibly) many.
- We started from the same routine but with specific features dropped. For a while we kept discretionary code.
- Interfacing wise something like discretionaries makes no sense: there is no intuitive way to use something equivalent (conditional content at page breaks) also because decisions are made as one goes before breaking kicks in. So it was dropped (less code due to less active/passive breakpoints).
- What did make sense, was exploring page shapes analogue to par shapes: heights (noflines), top-skip and bottomskip conditions kick in. Shape entries relate to what we call slots in columnsets.
- That set us on the path of trial runs and adaptive shapes. However this is done at the $\text{T}_{\text{E}}\text{X}$ end in loops that can test conditions and results and retry.
- At first we didn't want to do inserts but then realized that it was doable given some constraints: they are slot based.

- Because we want to work on the grid, we need to take care of half-line spacing and synchronize at breaks. This introduces top and bottom optional content, basically dumb discretionaries but done differently.
- In the end we also realized that trial runs made it possible to implement high performance balancing.

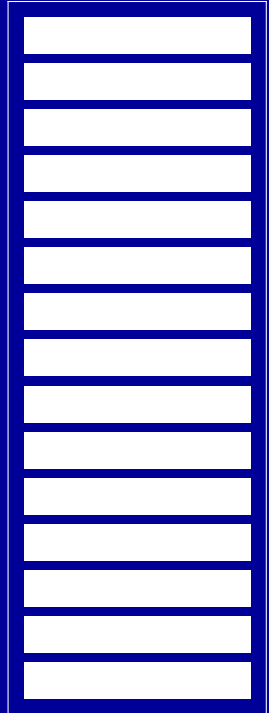
Activities: intercept

We have to intercept a ‘page stream’:

```
\beginmvl 1
  various content
\endmvl

\setbox\scratchboxone\flushmvl 1

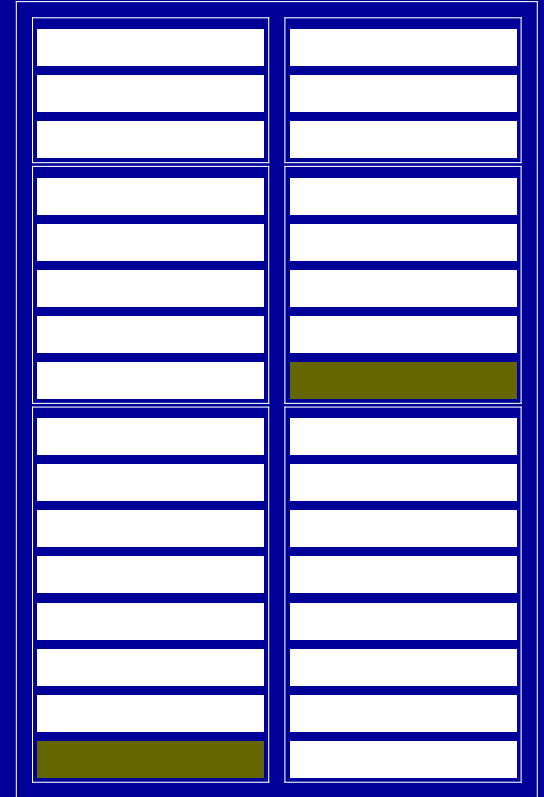
\beginmvl
  index 2
  % prevdepth 4pt
  options \numexpr "1 + "4\relax
\relax
  various content
\endmvl
```



Activities: shapes

```
\balancshape 3
      vsize      3\lineheight
      topskip    \strutht
      bottomsip  \strutdp
next
      vsize      5\lineheight
      topskip    \strutht
      bottomsip  \strutdp
next
      vsize      8\lineheight
      topskip    \strutht
      bottomsip  \strutdp
\relax

\setbox\scratchboxtwo\vbalance\scratchboxone
```

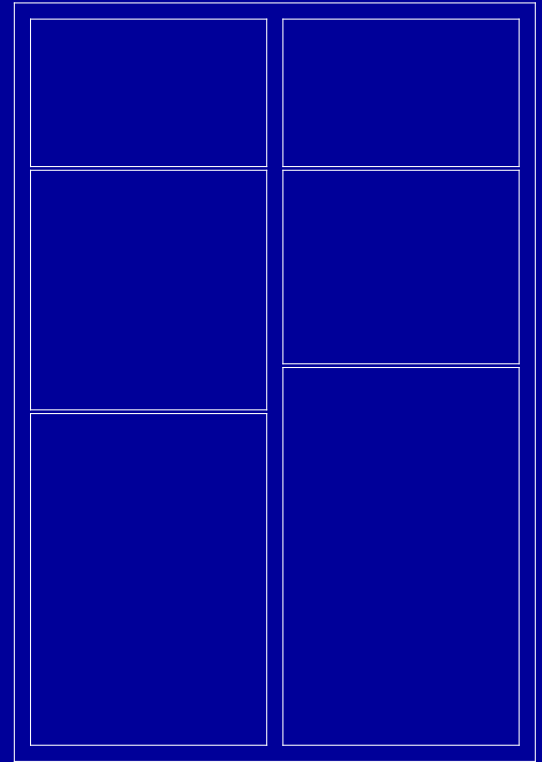


Activities: flush

```
\hbox \bgroup
  \localcontrolledendless {%
    \ifvoid\scratchboxtwo
      \expandafter\quitloop
    \else
      \setbox\scratchbox\ruledhbox\bgroup
        \vbalancedbox\scratchboxtwo
      \egroup
      \vbox to 12\lineheight \bgroup
        \box\scratchbox
        \vfill
      \egroup
      \hskip1em
    \fi
  }\unskip
\egroup
```

Activities: trials

```
\setbox\scratchboxtwo\vbalance\scratchboxone  
trial
```



Activities: breaks

```
\balanceboundary 3 1\relax  
\vskip\zeropoint  
\balanceboundary 3 0\relax  
\vskip\zeropoint  
\balanceboundary 3 0\relax
```

Using these with a proper callback we can go to a next slot. column, page or spread.

Activities: marks

Marks are kind of hidden in the resulting (split of) segments of the balanced list. They can be set using Lua calls:

```
node.direct.updatetopmarks()  
node.direct.updatefirstmarks()  
  
and  
  
node.direct.updatemarks(list)  
node.direct.updatefirstandbotmarks(list)
```

The pattern is:

- set the top marks
- loop over columns and set marks
- set the first marks

Activities: inserts

- We can assume a reasonable amount of notes.
- These are normally small with no (vertical) whitespace.
- Notes taking multiple lines may split.
- But we need to obey widow and club penalties.
- There can be math formulas but mostly inline.
- We need to keep them close to where they are referred from.
- We can ignore complex conflicting demands.
- As long as we get some result, we're fine.
- So users have to check what comes out.
- We don't assume fully automated unattended usage.

One can check for inserts with

```
<state> = \boxinserts <box>
```

and fetch them with

```
<box> = \vbalancedinsert <box> <class>
```

possible states are: `has_inserts`, `has_inserts_with_content`, `has_inserts_with_height`

Activities: discarding

We need to deal with (for instance) half line spacing on the grid which is a bit hackish but sort of generic. Kind of like this:

Trigger:

```
\par \vskip0pt \hrule discardable height 1sh depth 1sd width 1em \par
```

Before:

```
\vskip0pt \vbox discardable {\hpack{\strut BEFORE}} \par
```

After:

```
\vskip0pt \vbox discardable {\hpack{\strut AFTER}} \penalty -1 \par
```

Handling:

```
\vbalancddiscard 2          \relax % becomes a \nohrule
```

```
\vbalancddiscard 2 remove\relax % disappears
```

This mechanism is still evolving.

Activities: penalties

It is possible to discourage breaks at the end of a to be balanced list, like:

```
\balancefinalpenalties 6  
10000 9000 8000 7000 6000 5000  
\relax
```

Activities: passes

One can optimize the balancing with multiple passes, but we need to experiment a bit more with this. We just mention variables to play with:

- threshold
- tolerance
- looseness
- adjdemerits
- originalstretch
- emergencystretch
- emergencyfactor
- emergencypercentage

This model is simpler than the one used in the par builder but we can relatively easy extend it.

Some examples

We show some of the examples that we use for testing and figuring out additional features. We really start from usability, not from some abstract cq. utopian dream.